MASSIVELY SCALABLE APPS

LAB 3: Extra Knowledge about Docker

(For your own knowledge)

PREPARED BY:
MOHAMED MEDHAT

ARG VS ENV

1. **Scope & Availability:**
   - **ARG**: Only available during image BUILD time
   - **ENV**: Available during both BUILD and RUNTIME (container execution)
2. **How to use them**

```
# ARG can be set from command line during build
ARG VERSION=1.0
# Build with: docker build --build-arg VERSION=2.0 .

# ENV persists in the container
ENV API_KEY=12345
```

1. **Default Values:**
   - **ARG**: values can be overridden during build time
   - **ENV**: values persist in the container but can be overridden when starting the container
2. **Use Cases:**

```
# Good for build-time configuration
ARG JAVA_VERSION=17
FROM openjdk:${JAVA_VERSION}-slim
```

```
# Good for runtime configuration
ENV SERVER_PORT=8080
ENV SPRING_PROFILES_ACTIVE=prod
```

ENTRYPOINT VS CMD

1. **Primary Purpose:**
   - **ENTRYPOINT** defines the executable that will ALWAYS run when the container starts
   - **CMD** provides DEFAULT arguments to either the ENTRYPOINT or runs as a standalone command if ENTRYPOINT isn't specified
2. **Behavior with Command-Line Arguments:**
   - **ENTRYPOINT** arguments are NOT overridden when running the container with command-line arguments
   - **CMD** arguments ARE overridden by command-line arguments

ENTRYPOINT VS CMD

```
# Example 1: Using CMD alone
FROM ubuntu
CMD ["echo", "Hello World"]
```

When you run: docker run image → Output: Hello World
When you run: docker run image echo "Hi" → Output: Hi (CMD is completely overridden)

```
# Example 2: Using ENTRYPOINT alone
FROM ubuntu
ENTRYPOINT ["echo", "Hello"]
```

When you run: docker run image → Output: Hello
When you run: docker run image World → Output: Hello World # (Arguments are appended to ENTRYPOINT)

ENTRYPOINT VS CMD

```
# Example 3: Using both ENTRYPOINT and CMD
FROM ubuntu
ENTRYPOINT ["echo", "Hello"]
CMD ["World"]
```

When you run: docker run image → Output: Hello World
When you run: docker run image John → Output: Hello John # (CMD provides default argument to ENTRYPOINT)

Docker Networks

1. Network Basics

Docker networking enables communication between containers, and between containers and the outside world. Understanding networking is crucial for building scalable, secure containerized applications.

Default Networks

Docker automatically creates three networks:

- **bridge**: Default network for containers
- **host**: Removes network isolation between container and host
- **none**: No networking (isolated containers)

2. Network Drivers (bridge)

Bridge Network

- Default network driver
- Used when multiple containers need to communicate
- Works within a single host

```
# Create a bridge network
docker network create my-bridge

# Run container in network
docker run --network my-bridge nginx
```

2. Network Drivers (host)

Host Network

- Container shares host's networking namespace
- Best performance but less isolation

```
# Run container using host network
docker run --network host nginx
```

2. Network Drivers (overlay)

Overlay Network

- Enables communication between containers across multiple Docker hosts
- Essential for Docker Swarm and distributed applications

```
# Create overlay network
docker network create -d overlay my-overlay
```

2. Network Drivers (macvlan)

Macvlan Network

- Assigns MAC address to containers
- Makes them appear as physical devices on network

```
# Create macvlan network
docker network create -d macvlan --subnet=172.16.86.0/24
--gateway=172.16.86.1 -o parent=eth0 my-macvlan
```

3. Common Networking Commands

Network Management

```
# List networks
docker network ls

# Inspect network
docker network inspect bridge

# Create network
docker network create --driver bridge my-network

# Remove network
docker network rm my-network

# Remove all unused networks
docker network prune
```

3. Common Networking Commands

Container Networking

```
# Connect container to network
docker network connect my-network container1

# Disconnect container from network
docker network disconnect my-network container1

# Run container with specific IP
docker run --network my-network --ip 172.18.0.10 nginx
```

Docker Volumes

1. Volume Basics

Docker volumes provide persistent data storage and data sharing between containers. They are essential for maintaining data beyond container lifecycle.

Types of Storage

Volumes: Managed by Docker

Bind Mounts: Direct mapping to host filesystem

2. Working with Volumes

```
# Create volume
docker volume create my-vol

# List volumes
docker volume ls

# Inspect volume
docker volume inspect my-vol

# Remove volume
docker volume rm my-vol

# Remove all unused volumes
docker volume prune
```

2. Using Volumes in Containers

```
# Run container with volume
docker run -v my-vol:/app/data nginx

# Run container with bind mount
docker run -v /host/path:/container/path nginx

# Use read-only volume
docker run -v my-vol:/app/data:ro nginx
```

How to access ENV variables inside your code

```java
@Value("${MY_VARIABLE}")
private String myVariable;

// With default value
@Value("${MY_VARIABLE:defaultValue}")
private String myVariableWithDefault;
```

```java
private final Environment env;

@Autowired
public ClassConstructor(Environment env){
  this.env = env;
}
```

3. Common Use Cases

Database Storage

```
# PostgreSQL with persistent storage
docker run -d \
  --name postgres-db \
  -v postgres-data:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=secret \
  postgres
```

Application Logs

```
# Application with log volume
docker run -d \
  --name myapp \
  -v log-vol:/var/log/app \
  myapp
```

Shared Configuration

```
# Share configuration between containers
docker run -v config-vol:/etc/myapp container1
docker run -v config-vol:/etc/myapp container2
```

4. Best Practices

Volume Management

1. Use named volumes instead of anonymous volumes
2. Implement regular backup strategy
3. Clean unused volumes
4. Use appropriate volume drivers
5. Document volume usage

Security Considerations

5. Backing Up Volumes

Backing Up Volumes

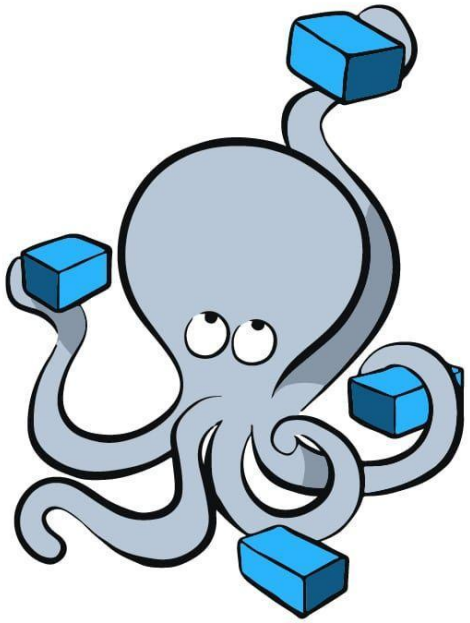
```
# Backup using temporary container
docker run --rm \
  -v my-vol:/source \
  -v $(pwd):/backup \
  alpine tar czf /backup/volume-backup.tar.gz /source
```

Restoring Volumes

```
# Restore from backup
docker run --rm \
  -v my-vol:/source \
  -v $(pwd):/backup \
  alpine tar xzf /backup/volume-backup.tar.gz -C /source
```

COMMON DOCKER COMMANDS

```
# Pull images from docker hub
docker pull image:tag

# Build images
docker build -t image:tag .

# Run image as container
docker run image_name
docker run -d --name container_name -p 8080:80 image_name

# Show running containers
docker ps

# Stop container
docker stop container_name

# Start a container
docker start container_name

# Remove container
docker rm container_name

# Remove all stopped containers
docker container prune

# Show images
docker images
```



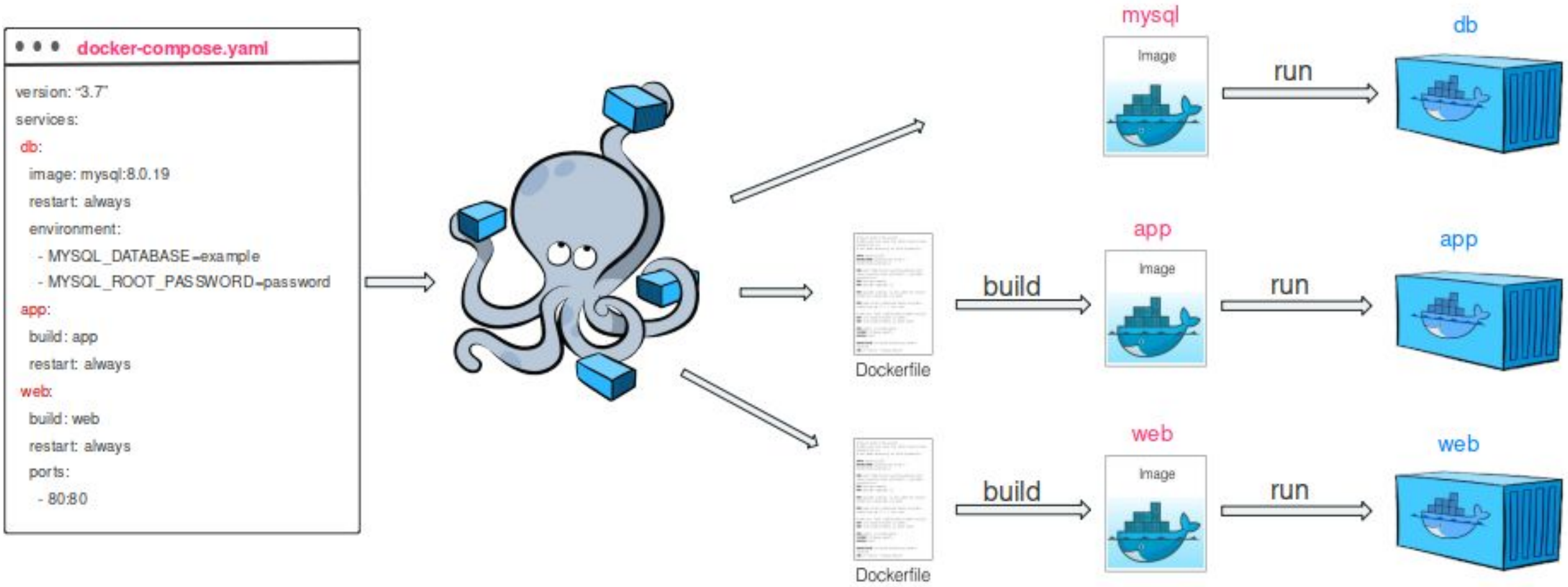
docker
Compose

DOCKER COMPOSE

- A tool for defining and running **multi-container applications**.
- It is the key to unlocking a streamlined and efficient **development** and **deployment** experience.

- It simplifies the control of your entire application stack,
  - making it easy to **manage services**, **networks**, and **volumes** in a **single, comprehensible YAML configuration file**.
  - Then, with a single command, you create and start all the services from your configuration file.

- Compose works in all environments; **production, staging, development, testing**, as well as CI workflows. It also has commands for managing the whole lifecycle of your application:
  - Start, stop, and rebuild services
  - View the status of running services
  - Stream the log output of running services
  - Run a one-off command on a service


docker-compose.yaml
version: "3.7"
services:
db:
image: mysql:8.0.19
restart: always
environment:
- MYSQL_DATABASE=example
- MYSQL_ROOT_PASSWORD=password
app:
build: app
restart: always
web:
build: web
restart: always
ports:
- 80:80
mysql
Image
run
db
Dockerfile
build
app
Image
run
app
Dockerfile
build
web
Image
run
web

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  webapp:
    image: nginx
    ports:
      - "80:80"
    networks:
        - docker_network


  database:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=secret
    volumes:
      - db-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  db-data:

networks:
  docker_network:
    driver: bridge
```

Docker compose version

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  webapp:
    image: nginx
    ports:
      - "80:80"
    networks:
        - docker_network


  database:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=secret
    volumes:
      - db-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  db-data:

networks:
  docker_netword:
    driver: bridge
```

Services defined as an array of containers

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  webapp:
    image: nginx
    ports:
      - "80:80"
    networks:
        - docker_network


  database:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=secret
    volumes:
      - db-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  db-data:

networks:
  docker_network:
    driver: bridge
```

Port mapping

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  webapp:
    image: nginx
    ports:
      - "80:80"
    networks:
        - docker_network


  database:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=secret
    volumes:
      - db-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  db-data:

networks:
  docker_netword:
    driver: bridge
```

Connecting to networks

Defining docker networks

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  webapp:
    image: nginx
    ports:
      - "80:80"
    networks:
        - docker_network


  database:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=secret
    volumes:
      - db-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  db-data:

networks:
  docker_network:
    driver: bridge
```

defining env variables

DOCKER COMPOSE CONFIG

```yaml
version: '3.9'

services:
  webapp:
    image: nginx
    ports:
      - "80:80"
    networks:
        - docker_network


  database:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=secret
    volumes:
      - db-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  db-data:

networks:
  docker_netword:
    driver: bridge
```

defining volumes and assigning volumes

USING DOCKERFILE IN DOCKER COMPOSE

In case you have not built your image yet, you can use build instead of image in the service

```yaml
version: "3.9"

services:
    app:
        build: . #path to your Dockerfile
        ...
```

HOW TO RUN MULTIPLE INSTANCES OF THE SAME SERVICE/IMAGE

1. Add Sufficient Port mappings to your docker compose, in case we need to create 10 instances we need to add 10 port mappings for example from 4040 to 4049

```yaml
services:
    app:
        ports:
            - "4040-4049:8080"
            ...
```

1. Use the --scale argument when running docker compose up

```
docker compose up -d --scale app=10
```

DOCKER COMPOSE COMMANDS

docker
Compose

```
# Start services
docker compose up

# Start in detached mode
docker compose up -d

# Stop services
docker compose down

# Stop and remove volumes
docker compose down -v

# View logs
docker compose logs

# Scale service
docker compose up -d --scale webapp=3

# List containers
docker compose ps
```

DOCKERIZING SPRING BOOT

```
# Base image
FROM openjdk:25-ea-4-jdk-oraclelinux9

# Set working directory
WORKDIR /app

# Copy the JAR file
COPY target/*.jar app.jar

# Expose the port your app runs on
EXPOSE 8080

# Command to run the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Build the project and ignore the tests
mvn clean package -DskiptTests

# Build docker image
docker build -t lab2-img .

# Run docker container
docker run -d --name lab2-container -p 8080:8080 lab2-img
```

THANK YOU